

Elements and Commands: the UI–DSL Correspondence

One signature, two dual carriers: how a UI element and a DSL command
are the same object read forwards and backwards

June 28, 2026

Abstract

The command companion makes a command a node of the free monad $\text{Free}\Omega A$ — program-as-data, a first-class value with three legs (parse, compose, interpret). This note shows that the user-interface tree is the *dual* of that object: the stateful UI is the cofree comonad $\text{Cofree}\Omega \text{St}$ over the *same* signature, an element-shaped tree annotated with state and behaviour. From this single fact the correspondence the informal account demands falls out rigorously. *Structurally*, an element’s type is a constructor of Ω , with its child-profile the constructor’s arity — one namespace worn three ways: UI type, command keyword, opcode tag. *Behaviourally*, an element is a Mealy node, and a command is a choice of event; the event loop is the *Free/Cofree pairing* that feeds the choice to the responder, and we prove this pairing is exactly the framework’s runtime step. Closing the loop, the interpretation map is a monoid isomorphism between closed commands (modulo behaviour) and the reachable UI state-changes — the precise form of “every state and event is a command, and conversely.” An element’s attributes are an option set, its overlay the options companion’s $\oplus$; a component with unfilled children is a template. We then say what the functional companion buys — routing as a **Traversable** descent, rendering as the comonadic **extend**, serialization as the parsing prism, the learned personalization agent as a closed program — and, with equal care, where it is *not* optimal: node identity lives off the functor, local high-frequency mutation defeats value semantics, observer graphs exceed trees, and the equational theory fixes *what* a program computes but not *when*. The boundary is exact: pure tree $\rightarrow$ free/cofree; finite sharing $\rightarrow$ a coalgebra; cycles, identity, or churn $\rightarrow$ the imperative.

Contents

1 Preliminaries: the objects to be joined	2
Part A. One signature, two dual carriers	3
2 The shared signature	3
3 Commands, and the Free/Cofree pairing	6
Part B. The element–command correspondence	8
4 The UI element, defined through the DSL	8
5 The behavioural bijection: every state-change is a command	8
6 Attributes are an option set; the overlay is the cascade	10

Part C. What the functional companion buys	11
7 Routing is a traversal; dispatch is ordered choice	11
8 Rendering is comonadic; serialization is a prism	12
9 One expressiveness ladder, four readings	13
10 The learned UI is a learned program	14
Part D. Calibration: where functional programming is not optimal	15
11 Identity lives off the functor	15
12 Local high-frequency mutation defeats value semantics	16
13 Observer graphs exceed trees	17
14 The theory fixes what, not when	17
15 The stratification: the right tool per layer	18
Part E. Synthesis	19
16 The correspondence, assembled	19
A Crosswalk: one object across the companions	21

1 Preliminaries: the objects to be joined

We recall, compactly but precisely, the four objects this note ties together, fixing notation and standing assumptions. Proofs of the recalled facts are in the cited companions; everything new is proved here.

The element-shape functor. A UI is a finite, typed, ordered tree of elements. The shape of that tree is governed by a single **signature functor**.

Definition 1.1 (Signature). Let $\mathbb{T}$ be a finite set of **element types**. The **signature functor** F is the polynomial functor

$$F X = \sum_{\tau \in \mathbb{T}} \left(\text{Attr}_{\tau} \times X^{\text{prof}(\tau)} \right),$$

one summand per type, carrying that type’s attribute record Attr_{τ} and a tuple of child-positions indexed by the type’s **profile** $\text{prof}(\tau)$ (a finite, possibly empty, position set). F is built from constants, $+$, $\times$, and exponentials by finite arity, hence *polynomial*; we assume it **Traversable** (Assumption 1.2). Each type’s **state space** $\text{St}(\tau)$ is a finite set assembled from atoms by product ($\times$), sum ($+$), and least fixed point (μ).

Assumption 1.2 (Standing assumptions). $\mathbb{T}$, every Attr_{τ} , and every $\text{St}(\tau)$ are finite; F is a polynomial **Traversable** functor; trees are finite and well-founded; and the concrete-syntax layout of each constructor (Proposition 8.2) is uniquely decodable. We work up to identity-isomorphism (renaming nodes changes no observable), except in Section 11, which is precisely about identity.

The UI tree. The bare shape is the initial algebra μF — the finite F -trees, i.e. the well-formed workspaces $\mathcal{T}$ of the editor chapter. An editor acts on μF by the structural primitives $\mathcal{O}_\top = \{\text{mint}, \text{attach}, \text{detach}, \text{delete}, \text{relabel}\}$, and we recall its central property; the **runtime** layer instead drives node state via a Mealy handler per type, $\text{handle}_\tau : \text{St}(\tau) \times \text{Ev} \rightarrow \text{St}(\tau)$, over a finite event alphabet Ev .

Theorem 1.3 (Generative completeness; recalled). *The structural primitives are universal: from the seed $W_\emptyset$ they reach exactly $\mathcal{T}$, and $(\mathcal{T}, \rightarrow)$ is strongly connected. Behaviourally, for each type the runtime relation generated by handle_τ reaches all of $\text{St}(\tau)$ from its initial state (with edit mode supplying any state a handler omits). Consequently every valid UI configuration is reachable, and the joint system is finite, hence decidable to analyse.*

The command. The command companion fixes the **command signature** Ω — one operator per name, of arity its parameter profile — and the free monad

$$\text{Free}\Omega A = \mu X. (A + \Omega X),$$

the $\text{free}\langle\Omega, A\rangle$ carrier: a **command is a node of** $\text{Free}\Omega A$, first-class because it is a value in a **Monad**. A **closed** command ($A = \emptyset$) lies in $\text{Free}\Omega \emptyset \cong \mu\Omega$, a runnable program. Three maps act on this one term: $\text{parse} : \Sigma^* \rightarrow \text{Free}\Omega A$ (build, an unfold), $\text{compose} : \text{Free}\Omega \emptyset \rightarrow \Sigma^*$ (serialize, a fold), and $\llbracket - \rrbracket : \text{Free}\Omega \emptyset \rightarrow T(R)$ (interpret, a fold into an effect monad T).

Options. An **option** is a key–value pair $o = (k(o), v(o))$ with $k(o)$ in a key space; an **option set** O has injective key map, so $k(\cdot) : O \cong K(O)$ and $O[k]$ denotes the option at key k . The companion’s algebra $(\oplus, \sim, \text{diff})$ gives: the **precedence overlay** $O_1 \oplus O_2$ (keep all keys, O_2 wins on overlap), an associative monoid with identity $\emptyset$; **agreement** $O_1 \sim O_2$ (identical on shared keys), a reflexive–symmetric tolerance (not transitive); and the projected **difference** diff_π . Overlay is commutative exactly on agreeing sets.

The functional carriers. We use the companion’s carriers **maybe**, **result**, **pair/kv_pair** (the environment comonad), **view/producer** (lazy sequences), and the two that organise this note: $\text{free}\langle F, A \rangle = \text{Free}F A$ (a **Monad**, program-as-data) and its dual $\text{cofree}\langle F, C \rangle = \text{Cofree}F C$ (a **Comonad**, an annotated tree). The protocols invoked are **Functor**, **Applicative**, **Selective**, **Monad**, **Traversable**, **Foldable**, **Alternative**, and **Comonad**.

Remark 1.4 (The thesis in one line). Free and Cofree over the *same* F are categorical duals. The command companion develops the Free side; the UI framework, unknowingly, develops the Cofree side. Naming that duality is the whole content of the correspondence: an element and a command are one signature read backwards and forwards.

Part A. One signature, two dual carriers

2 The shared signature

The first move is to observe that the element types and the command names are not two alphabets to be related but one alphabet seen twice.

Definition 2.1 (Identification of signatures). Identify the command signature with the element signature, $\Omega = F$, by sending each element type $\tau \in \mathbb{T}$ to the command constructor of the same name and arity $\text{prof}(\tau)$. Under this identification a constructor is at once: an element **type** (a node kind of the UI tree), a command **name** (a production keyword of the DSL), and — via the opcode companion — an **opcode** (a machine word). The constructor’s arity is read three ways: the type’s **child-profile**, the production’s right-hand side, and the command’s **parameter profile**.

Proposition 2.2 (Types are constructors). *The identification $\Omega = F$ is a bijection on constructors that preserves arity, and it is forced once one asks that building an element and naming its command use the same data. In particular $\mu F = \mu \Omega$ as sets of finite trees: a workspace shape is a closed first-order command term, and conversely.*

Proof. Both signatures are sums indexed by a name set with a finite position tuple per summand; the identification matches names and, by Definition 2.1, positions, so it is an arity-preserving bijection of polynomial functors. Initial algebras of equal functors are equal, giving $\mu F = \mu \Omega$. The forcing is the observation that a node’s children and a command’s parameters are the same finite indexed family: to build the node is to supply exactly the data the command names. $\square$

Remark 2.3 (Three coordinates name one thing). The opcode companion encodes the constructor *tag*; the parse/compose pair relates the surface *keyword* to that tag; so a single element/command is pinned by three coordinates — keyword (concrete syntax), constructor (abstract syntax), opcode (machine word) — with **parse**, **compose** the left edge and opcode assignment the right. The element’s *type* is the same constructor, making the UI node the fourth reading of the one tag. The polarity pairs, product attributes, and hierarchies the opcode companion encodes (PAIR, PROD, TREE) are exactly the $+$, $\times$, μ of the state algebra St : flat attributes are a Π -product (PROD), a containment hierarchy a dependent Σ -product (TREE).

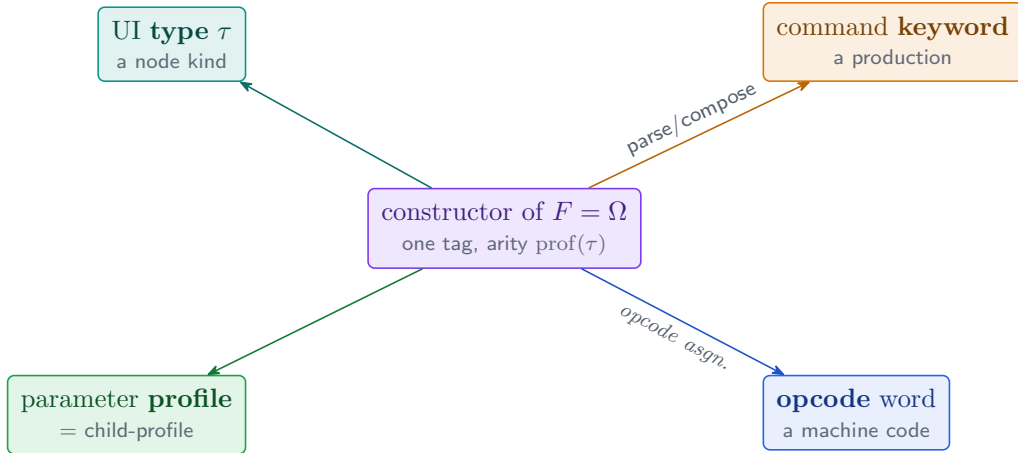


Figure 1: **Four faces of one constructor.** A single tag of the shared signature $\Omega = F$ is read as a UI type, a DSL keyword, an opcode, and a parameter/child profile; **parse/compose** is the keyword $\leftrightarrow$ tag edge and opcode assignment the tag $\leftrightarrow$ word edge (Definition 2.1 and Remark 2.3).

The bare tree μF has no state. The running UI annotates every node with its current state (and, below, its behaviour). That annotated tree is a cofree comonad.

Definition 2.4 (The stateful UI tree). The **stateful UI tree** is the carrier

$$U = \mu X. (\text{St} \times F X),$$

the finite F -trees in which every node v additionally carries an annotation $\sigma(v) \in \text{St}(\tau(v))$. It is the well-founded fragment of the cofree comonad $\text{Cofree } F \text{ St}$, and carries the comonad operations

$$\text{extract} : U \rightarrow \text{St} \quad (\text{the root's annotation}), \quad \text{duplicate} : U \rightarrow \mu X. (U \times F X),$$

where **duplicate** relabels each node v by the entire sub-tree rooted at v ; **extend** $g = F(\dots) \circ \text{relabel}$ applies $g : U \rightarrow B$ at every node to its own sub-tree, **extend** $g : U \rightarrow \mu X. (B \times F X)$.

Proposition 2.5 (U is a comonad; the laws are the UI's locality). (**extract**, **duplicate**) *satisfy the comonad laws* $\text{extract} \circ \text{duplicate} = \text{id}$, $F(\text{extract}) \circ \text{duplicate} = \text{id}$ (after the evident reassociation), and coassociativity. Reading them in UI terms: **(i)** the root sub-tree's root is the root (**extract** after **duplicate**), so a node's annotation is recoverable from its context; **(ii)** annotating each node with its own singleton context is the identity; **(iii)** contexts of contexts agree with contexts (coassociativity), so a context-dependent pass may be staged. Hence any rendering or layout whose output at a node depends on that node's sub-tree is a well-defined **extend**.

Proof. These are the standard cofree-comonad laws for the functor $X \mapsto \text{St} \times F X$, which hold for any F ; on the well-founded (μ) fragment the relabelings terminate, so the operations are total. The UI readings are the laws transcribed under Definition 2.4: **duplicate** places at v the sub-tree at v , whose **extract** is $\sigma(v)$, giving (i); **extend extract** = **id** gives (ii); coassociativity is the equality of the two ways to form contexts-of-contexts, (iii). $\square$

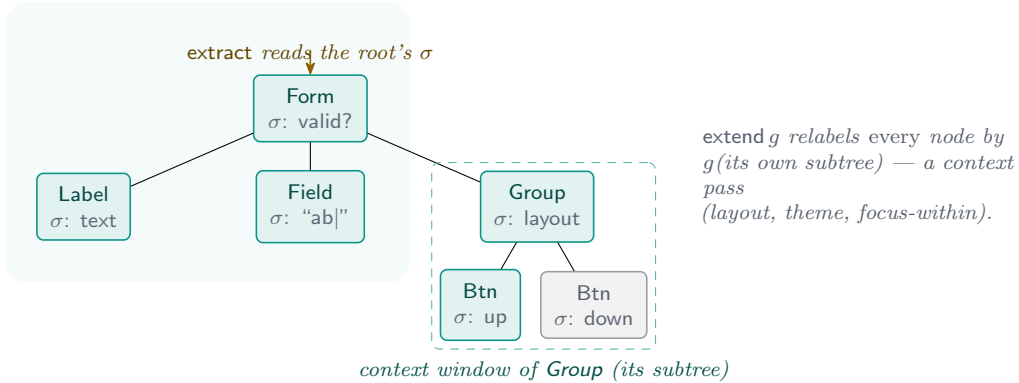


Figure 2: **The stateful UI is a cofree comonad.** An element-shaped tree (F -shape) with a state annotation σ at every node: the carrier $U = \mu X. (\text{St} \times F X)$ of Definition 2.4. **extract** reads a node's own state; **extend** recomputes each node from its subtree context (the dashed window) — the comonadic form of context-dependent layout and rendering (Propositions 8.1 and 2.5).

Definition 2.6 (Behavioural annotation; the element as a Mealy node). Annotate each node additionally with its handler. Currying the Mealy map presents a node of type τ as a **response product**

$$\widehat{\text{handle}}_\tau : \text{St}(\tau) \longrightarrow \prod_{e \in \text{Ev}} (\text{St}(\tau) + \{\perp\}), \quad \widehat{\text{handle}}_\tau(s)(e) = \begin{cases} \text{handle}_\tau(s, e), & e \in E(\tau), \\ \perp, & \text{otherwise,} \end{cases}$$

i.e. for each current state a chosen next-state *for every event* (absent where unhandled). The **behavioural UI** is the cofree tree $\widehat{U} = \text{Cofree}F(\text{St} \times \prod_e(\text{St} + \{\perp\}))$ carrying both state and response product at each node. An **element** is one node of $\widehat{U}$: a type, an attribute record, a state, and a response product.

The response product is the cofree side's payload. The next section exhibits its dual — a command — and the operation that pairs them.

3 Commands, and the Free/Cofree pairing

A command is a node of $\text{Free}\Omega A$ (Section 1). Where a handler node *provides* a response for every event (a product), a command *selects* one (a sum). Product pairs with sum; the pairing is application, and it is the event loop.

Definition 3.1 (Located command; event selection). An **atomic located command** is a pair (p, e) of a path p to a node of U and an event $e \in \text{Ev}$ (with any payload folded into e); equivalently it is the **choice**, at the addressed node, of one coordinate of that node's response product. A **command program** is a node of $\text{Free}\Omega A$ whose constructors are located commands — a free-monad sequence/branch of selections; a closed program lies in $\text{Free}\Omega \emptyset$. Authoring commands ($\text{mint}, \dots, \text{relabel}$) are the located commands whose effect is structural rather than on σ .

Definition 3.2 (The pairing / run). Define $\text{run} : \text{Free}\Omega \emptyset \times U \rightarrow U$ by structural recursion on the program. On an atomic located command (p, e) at the node $v = U@p$,

$$\text{run}((p, e), U) = U[v := \widehat{\text{handle}}_{\tau(v)}(\sigma(v))(e)] \quad \text{if } e \in E(\tau(v)), \quad \text{else } U,$$

i.e. replace the addressed node's annotation by the selected response, leaving shape and all other annotations fixed; on a sequence $c_1; c_2$, $\text{run}(c_1; c_2, U) = \text{run}(c_2, \text{run}(c_1, U))$; on an authoring command, apply the corresponding structural primitive. Equivalently $\text{run} = \llbracket - \rrbracket$ with the effect monad T the State monad over U , $R = \mathbf{1}$.

Theorem 3.3 (The pairing is the runtime step). *For an atomic located command (p, e) routing to node v , $\text{run}((p, e), -)$ equals the framework's runtime step $\xRightarrow{e}$ that routes e to v ; for an authoring command it equals the corresponding editor step $\rightarrow$. Hence on programs, run is the sequential composition of runtime and authoring steps, and the interpretation $\llbracket - \rrbracket$ of a closed command is the joint UI transition it denotes.*

Proof. The runtime step, by definition, routes e to the addressed node v and updates its state by $\sigma(v) \mapsto \widehat{\text{handle}}_{\tau(v)}(\sigma(v), e)$ when $e \in E(\tau(v))$ (and is the identity otherwise), changing nothing else; this is exactly the right side of Definition 3.2 after uncurrying $\widehat{\text{handle}}$. Authoring constructors are interpreted as their structural primitives by construction. Sequencing in the free monad is interpreted as composition in the State monad (the behaviour chapter's $f_{c_2} \bullet f_{c_1}$), which is the composition of the corresponding steps. Therefore $\text{run} = \llbracket - \rrbracket$ agrees step-for-step with the joint transition relation. $\square$

Remark 3.4 (Why this is a *pairing*). The content is the duality *product pairs with sum*: a handler node is a product over events (provide a response to each), a command is a sum over events (choose one), and run is the canonical evaluation $(\prod_e R_e) \times (\sum_e \mathbf{1}) \rightarrow R$ applied node-wise. Extended along the recursion, this is the standard pairing of a **cofree**-comonad of handlers with a **free**-monad of commands: the UI is the comonad that *responds*, the command is the monad that *drives*, and the

event loop is their pairing. The framework's two transition systems (runtime and authoring) are the two halves of this one operation — state-moves and shape-moves — because $\Omega = F$ carries both.

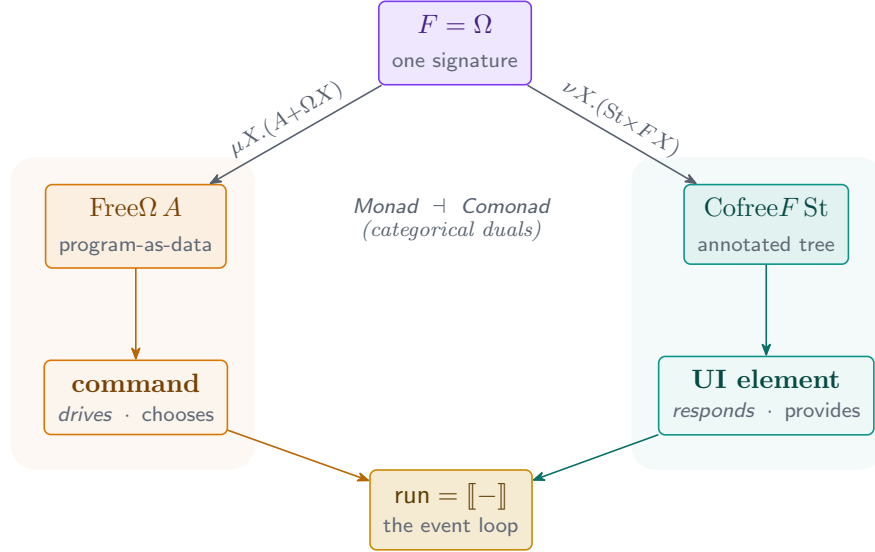


Figure 3: **One signature, two dual carriers.** The command companion develops the free monad $\text{Free}\Omega A$ (it *drives*: a program that chooses); the UI framework develops the cofree comonad $\text{Cofree}F St$ (it *responds*: a tree that provides). Both are built over the same $F = \Omega$, and the event loop $\text{run} = [-]$ is their pairing (Theorem 3.3).

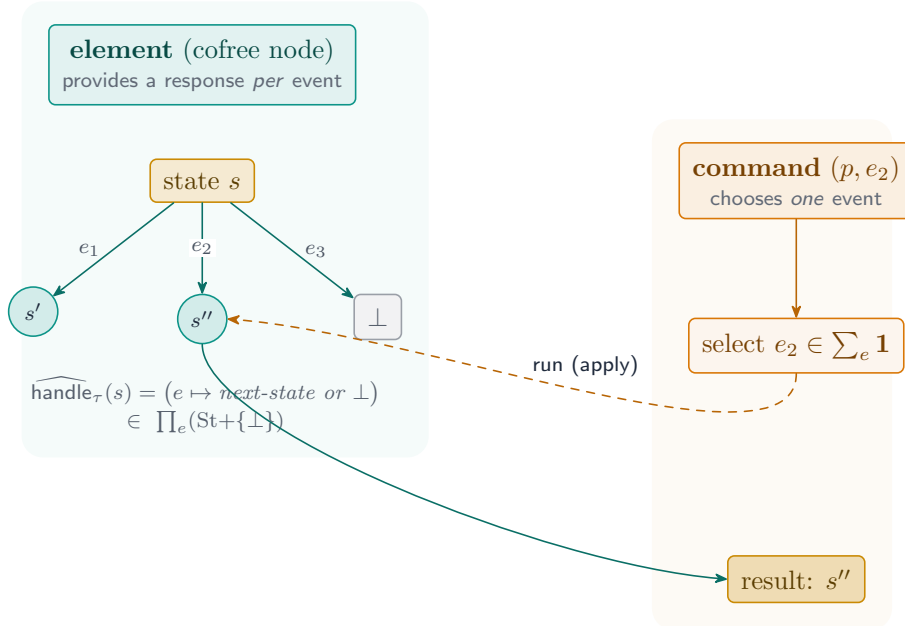


Figure 4: **Product pairs with sum: the pairing is application.** A handler node is a *product* over events (a next-state for each); a command is a *choice* of one event (a summand). The run feeds the choice to the product and reads off the selected response — exactly the runtime step (Theorem 3.3 and Remark 3.4).

Part B. The element–command correspondence

4 The UI element, defined through the DSL

We can now give the element its definition — not as a primitive, but through its four relations to the surrounding framework.

Definition 4.1 (UI element; component). A **UI element** is a node of the behavioural UI tree $\widehat{U}$ (Definition 2.6), i.e. a tuple

$$\delta = (\tau, \text{attrs}, \sigma, \widehat{\text{handle}}_\tau)$$

read through four faces:

- (1) **(DSL face)** τ is a *constructor* of Ω of arity $\text{prof}(\tau)$ — the element’s command name / production / opcode, with child-profile the parameter profile (Definition 2.1);
- (2) **(option face)** attrs is an *option set* keyed by attribute name — equally a command *binding* (Section 6);
- (3) **(state face)** $\sigma \in \text{St}(\tau)$ is a point of the $\times / + / \mu$ state algebra;
- (4) **(behaviour face)** $\widehat{\text{handle}}_\tau$ is a *Mealy node* — the cofree annotation, and the *interpret-leg* of the element’s commands (Theorem 3.3).

A **component** is a composite element: an internal node of $\widehat{U}$, hence a sub-term of the command tree. A component with one or more unfilled child positions is a *template* over Ω (a node of $\text{Free}\Omega A$ with holes A); filling them is instantiation $t \gg (u \triangleleft d)$ (Section 9).

Remark 4.2 (Atomic vs. composite, once more). An **atomic element** is a leaf: a nullary constructor of Ω , a *bare command*. A **composite element** is an internal node: a constructor of positive arity, a command carrying sub-commands. Nesting is composition; there is no separate “container” primitive, exactly as there is no separate composite command — both are just internal nodes of the one tree (Proposition 2.2).

5 The behavioural bijection: every state-change is a command

The informal demand is that “all elements map to a DSL command for all states/events, and conversely.” Taken naively — one element, one command — it is a category error: an element admits many commands (one per event, plus authoring). The correct statement separates the two axes and is exact on each.

Theorem 5.1 (Behavioural bijection). *Let $\mathcal{C}^0 = \text{Free}\Omega \emptyset$ be the closed commands, and let $c \approx c'$ mean $\llbracket c \rrbracket = \llbracket c' \rrbracket$ as transformations of $\widehat{U}$. Let $\text{Trans}(\widehat{U})$ be the monoid (under composition) of UI state-changes generated by the runtime events and authoring primitives. Then $\llbracket - \rrbracket$ descends to a monoid isomorphism*

$$\llbracket - \rrbracket : \mathcal{C}^0 / \approx \xrightarrow{\cong} \text{Trans}(\widehat{U}).$$

Equivalently: every UI state-change is realised by a closed command, distinct state-changes by inequivalent commands, and command sequencing is transition composition. This is the precise sense of “every state and event is a command, and conversely.”

Proof. Well-defined and injective. By construction $\approx$ is the kernel of $\llbracket - \rrbracket$, so the induced map on $\approx$ -classes is well-defined and injective.

Homomorphism. The behaviour chapter interprets a program $c_1; c_2$ as $\llbracket c_2 \rrbracket \bullet \llbracket c_1 \rrbracket$, composition in the State monad; thus $\llbracket c_1; c_2 \rrbracket = \llbracket c_2 \rrbracket \circ \llbracket c_1 \rrbracket$, and $\llbracket - \rrbracket$ carries free-monad sequencing to composition, with the empty program to id.

Surjectivity (the “conversely”). $\text{Trans}(\widehat{U})$ is generated by two families: each runtime event e routed to each node, and each authoring primitive. By Theorem 3.3 the former is $\llbracket (p, e) \rrbracket$ for the located command addressing that node, and the latter is $\llbracket c \rrbracket$ for the corresponding authoring constructor; so every *generator* of $\text{Trans}(\widehat{U})$ lies in the image. A monoid homomorphism whose image contains a generating set is onto the generated monoid. Hence $\llbracket - \rrbracket$ is surjective, and with injectivity an isomorphism.

Coverage of all states. That the generators actually reach every configuration — so $\text{Trans}(\widehat{U})$ acts transitively on the valid states — is generative completeness (Theorem 1.3): authoring reaches every shape and is strongly connected, and the per-type runtime reaches every state. Thus not only is each *move* a command, but the commands suffice to reach *every* state from any other. $\square$

Corollary 5.2 (The two correct readings). *The correspondence holds on two distinct axes, and only these:*

- (1) structural — element **types** $\leftrightarrow$ command **constructors**, an arity-preserving bijection (Proposition 2.2);
- (2) behavioural — UI **state-changes** $\leftrightarrow$ closed **commands** modulo behaviour, a monoid isomorphism (Theorem 5.1).

The naive “element $\leftrightarrow$ command” conflates a constructor with a closed term over it; resolved into these two axes, “everything is a command” is literally true.

Proof. Immediate from the cited results; the conflation is the type error of equating an element type (a generator) with an individual program built from it. $\square$

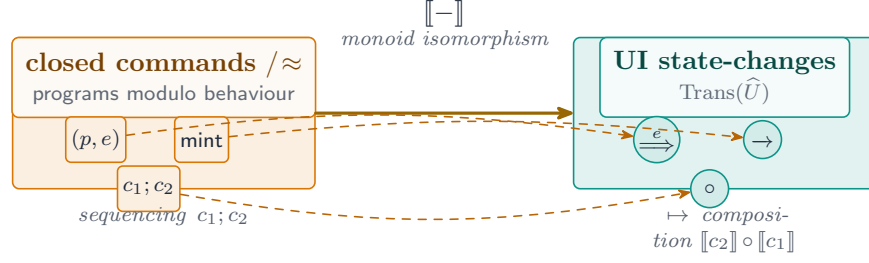


Figure 5: **The behavioural bijection (Theorem 5.1).** Interpretation $\llbracket - \rrbracket$ sends closed commands modulo behaviour *isomorphically* onto the reachable UI state-changes: generators (events, edits) map to generators, sequencing to composition. “Every state and event is a command, and conversely” — surjectivity is generative completeness, injectivity is the definition of $\approx$.

6 Attributes are an option set; the overlay is the cascade

The option face of Definition 4.1 is an equality, not an analogy.

Theorem 6.1 (Attributes/binding/options coincide). *For any element, the following are the same object: its **attribute record**, the **binding** of its constructor’s command, and an **option set** keyed by attribute/parameter name. Under this identification:*

- (1) *setAttr and setStyle are the precedence overlay: applying a patch P is $\text{attrs} \mapsto \text{attrs} \oplus P$;*
- (2) *a **style cascade** (defaults, then theme, then inline) is the left fold $A_{\text{def}} \oplus A_{\text{theme}} \oplus A_{\text{inline}}$, well-defined and order-consistent (“later wins”) since (option sets, $\oplus$, $\emptyset$) is a monoid;*
- (3) *two attribute sources are co-applicable in any order iff they agree ($\sim$) — the commutativity-iff-agreement law, identical to the modifier-commutativity law for command behaviour;*
- (4) *the change set the reconciler computes between UI states s, s' is $\text{diff}_\pi(\text{attrs}(s), \text{attrs}(s'))$, with π selecting declared value, default, or value-or-default.*

Proof. A command binding assigns to each parameter (a unique key) a value-or-absence $\beta_c(a) \in D_a \cup \{\perp\}$, with β_c injective on the profile; this is exactly an option set keyed by the profile (synthesis chapter). An attribute record is the same finite key→value map keyed by attribute name; the three views differ only in the name of the index set. The four clauses are then the options companion’s laws transported verbatim: (1) is the definition of $\oplus$ as “ O_2 overrides O_1 on shared keys”; (2) is associativity with identity $\emptyset$; (3) is the proposition that $O_1 \oplus O_2 = O_2 \oplus O_1 \iff O_1 \sim O_2$, which the behaviour chapter shows coincides with pairwise commutativity of the corresponding modifiers; (4) is the definition of diff_π . $\square$

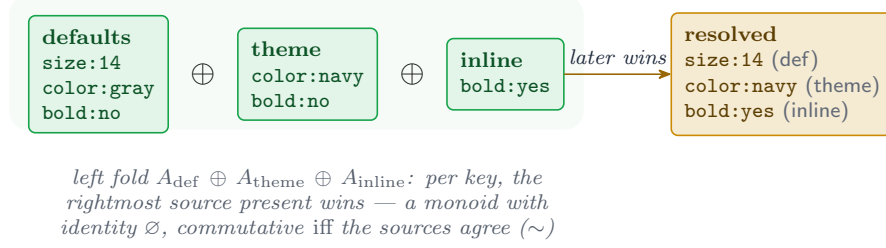


Figure 6: **The style cascade is the option overlay $\oplus$.** Defaults, theme, and inline attributes are option sets; cascading them is the left fold of the precedence overlay (“later wins”), keyed per attribute. This is literally the options companion’s monoid and the command’s sequential override (Theorem 6.1 and Remark 6.2).

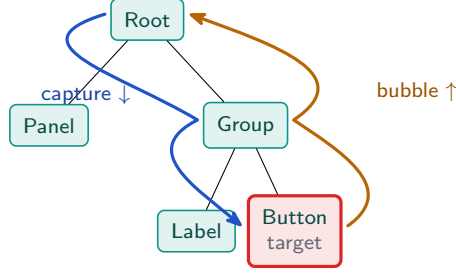
Remark 6.2 (One resolution device everywhere). “Later wins” (style cascade, sequential override), “leftmost wins” (committed event dispatch, PEG choice), and “highest precedence wins” (opcode tactic) are the single device of a total order making a nondeterministic combine deterministic, instantiated at apply-time, parse-time, and encode-time. Attribute cascading is this device on the option monoid; the next part shows event dispatch is the same device on the parser’s **Alternative**.

Part C. What the functional companion buys

7 Routing is a traversal; dispatch is ordered choice

Proposition 7.1 (Routing as a Traversable descent). *Hit-testing and event dispatch are one Traversable traversal of F with **Alternative** selecting the consuming node. Concretely, routing an event is the effectful descent traverse over the node’s children that (i) by the disjoint-region geometry visits the unique child whose region contains the pointer (or, for keyboard events, follows the focus path), and (ii) at each level offers the local handler via left-biased alt: the first ancestor-or-node whose handler is defined on the event consumes it. The two dispatch phases — capture ($\text{root} \rightarrow \text{target}$) and bubble ($\text{target} \rightarrow \text{root}$) — are the pre-order and post-order of the same traversal.*

Proof. F is **Traversable** (Assumption 1.2), so traverse threads an effect across a constructor’s children in a fixed order; geometry makes the pointer-containing child unique (the editor’s disjoint-sibling condition), so the descent is deterministic. Left-biased alt is, by the parser companion, exactly committed ordered choice: $\text{alt } p q$ runs p , and only on its failure q ; taking p to be “the deeper handler” and q “the shallower” realises “first defined handler wins,” i.e. event capture; reversing the bias gives bubbling. Pre-/post-order are the two threadings of one **Traversable** structure. $\square$



one **Traversable** descent; **Alternative** (left-biased) picks the first handler that consumes — the committed choice of PEG and override

Figure 7: **Routing is a traversal; dispatch is ordered choice (Proposition 7.1).** Geometry/focus fixes a unique root→target path. The two dispatch phases are the pre-order (capture, ↓) and post-order (bubble, ↑) of one **Traversable** descent; “first handler wins” is left-biased **Alternative**.

Remark 7.2 (Dispatch precedence is the resolution device). Thus event precedence (which handler fires) is not UI-specific machinery but the **Alternative** monoid’s alt/aempty — the same committed choice as PEG parsing and the dual, at run-time, of sequential override (Remark 6.2). That **result** is *not* **Alternative** is why an error-reporting dispatcher must elect an explicit merge rather than inherit a free “no handler.”

8 Rendering is comonadic; serialization is a prism

Proposition 8.1 (Rendering as extend). *A render or layout pass whose output at a node depends on that node’s sub-tree (intrinsic sizing, theme inheritance, focus-within) is the comonadic extend $g : U \rightarrow U'$ for $g : U \rightarrow B$ reading the local context; extract reads a node’s own value. An incremental render is extend restricted to a dirty sub-comonad: nodes whose context is unchanged keep their previous output by the comonad law $\text{extend extract} = \text{id}$.*

Proof. By Proposition 2.5, a context-dependent assignment is a well-defined extend; extract is the local read. If g depends only on an unchanged sub-context at v , then g at v is unchanged, so re-running $\text{extend } g$ there reproduces the prior value — formally, on the clean sub-comonad g factors through extract, where $\text{extend extract} = \text{id}$. $\square$

Proposition 8.2 (Serialization/addressing as the parsing prism). *The UI tree round-trips through the DSL: $\text{compose} = \text{cata}[\text{print}] : \mu F \rightarrow \Sigma^*$ is a total injection, and $\text{parse} = \text{compose}^{-1} : \Sigma^* \rightarrow \mu F$ its partial inverse on $L = \text{im}(\text{compose})$, satisfying the prism laws $\text{parse}(\text{compose } t) = t$ and $s \in L \Rightarrow \text{compose}(\text{parse } s) = s$. Consequently every element and every UI state has a **canonical command surface** naming it, and conversely each surface in L denotes a unique UI.*

Proof. F is a polynomial **Traversable** functor with a uniquely decodable layout (Assumption 1.2); the parsing companion then gives compose as the unique fold, total and (by decodability) injective, and parse as its anamorphic inverse on the image, with the prism laws. Applying compose to

a sub-tree yields the surface naming that element; applying it to the state-decorated tree (the decorated grammar) names a state. Injectivity gives uniqueness in both directions. $\square$

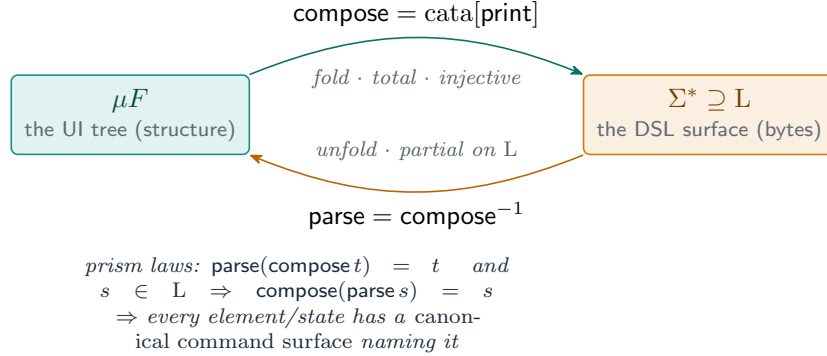


Figure 8: **Serialization is a prism; the UI is addressable (Proposition 8.2).** `compose` folds a UI tree to its DSL surface (total, injective); `parse` unfolds it back on the recognized language L . The round-trip laws make “name this element/state by a command” well-defined — the bridge that lets a closed command in Theorem 5.1 denote a state-change concretely.

Remark 8.3 (Addressability is the bridge to Part B). Proposition 8.2 is what lets a *closed command* in Theorem 5.1 *name* a state-change concretely: `parse` the surface to a command term, run it by the pairing. The grammar is the UI’s declarative image; the pairing is its operational one; the prism is the isomorphism between them on L .

9 One expressiveness ladder, four readings

Proposition 9.1 ($\text{Free}_{\text{Ap}} \subseteq \text{Free}_{\text{Sel}} \subseteq \text{Free}$, read four ways). *The free-construction ladder grades, by exactly the $\times, +, \mu$ strata, four notions at once:*

	Free_{Ap}	Free_{Sel}	Free
<i>parser</i>	<i>fixed shape</i>	<i>static guard</i>	<i>value-driven shape</i>
<i>template</i>	<i>fill-in-the-blanks</i>	<i>conditional section</i>	<i>recursive/data-driven</i>
<i>command</i>	<i>independent args</i>	<i>flag gates a branch</i>	<i>arg chooses sub-command</i>
<i>UI element</i>	<i>static layout</i>	<i>conditional child</i>	<i>data-driven subtree</i>

The inclusions are strict and the same in all four columns: no value-inspection (applicative), static branch only (selective), a resolved value chooses the substructure (monadic, the μ case).

Proof. The ladder is the parser/template grading of the companions; Definition 2.1 makes the UI element’s child-shape an Ω -term, so a child position fixed in advance is applicative, a child present-or-absent under a static guard is the selective **when**, and a child sub-tree chosen by a resolved value is the monadic $\gg=$ over μ . Strictness is inherited from the companions: a value that selects shape is not expressible without $\gg=$. $\square$

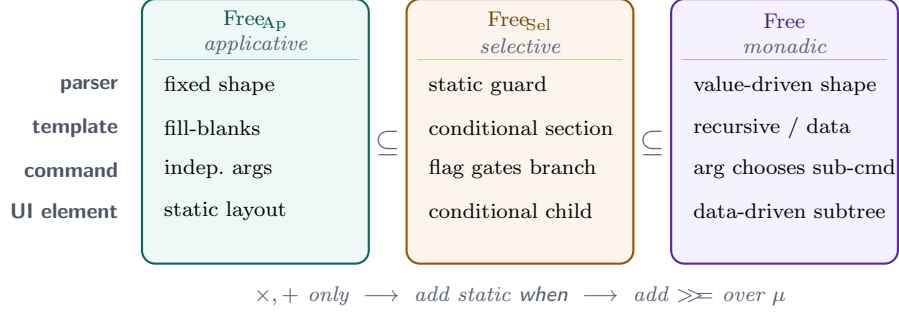


Figure 9: **One ladder, four readings (Proposition 9.1).** The free-construction chain $\text{Free}_{\text{Ap}} \subseteq \text{Free}_{\text{Sel}} \subseteq \text{Free}$ grades parser, template, command, and UI expressiveness identically: no value-inspection (applicative), a static branch (selective), a resolved value choosing the substructure (monadic, the μ case).

Remark 9.2 (Where each UI idiom sits). A static form is applicative; “show this panel only if logged in,” with both states pre-built, is selective; a tree/list whose node count is a runtime value, or a wizard whose next step depends on an answer, is monadic. The selective rung is the corpus’s guarded action $\gamma(w, e) = \text{when}$ at term level; the monadic rung is where genuine data-dependence — and the cost discussion of Part D — begins.

10 The learned UI is a learned program

The personalization layer now reads in command terms, tightening its safety guarantees to statements about $\llbracket - \rrbracket$.

Proposition 10.1 (Glamour as closed program; agent as interpreter). *Let $\Omega_\kappa \subseteq \Omega$ be the sub-signature of capability-permitted constructors. The preemptive agent’s **glamour** is a closed program $g \in \text{Free}\Omega_\kappa \emptyset$; the agent is the free-monad interpreter $\llbracket - \rrbracket$ run as a pre-pass on the default tree. Then:*

- (1) (safety) $\llbracket g \rrbracket$ preserves every shipped invariant — schema, no-dangling, protection — because each Ω_κ -constructor does and invariant-preservation composes; so $\llbracket g \rrbracket(\hat{U})$ is valid;
- (2) (reversibility) $\llbracket g \rrbracket$ is undoable and reset-to-default is reachable, by strong connectivity (Theorem 1.3);
- (3) (optimality target) compiling the learned utility’s per-context optimum into g makes $\llbracket g \rrbracket$ drive the default to that optimum, when it is κ -reachable.

Hence “personalize the UI” is “synthesize and run a closed command”; the learned model emits a program, and the lattice of invariants bounds it.

Proof. A glamour is a guarded rewrite by κ -permitted operations, i.e. a finite located-command program over Ω_κ , hence a node of $\text{Free}\Omega_\kappa \emptyset$; running it is $\llbracket - \rrbracket$ (Definition 3.2). (1) Each permitted constructor is a step of the invariant-restricted editor, preserving schema and (with the global-reset convention) no-dangling; preservation of a meet of invariants is the conjunction of preservations, so the composite program preserves all of them. (2) The unrestricted editor is strongly connected (Theorem 1.3), so from $\llbracket g \rrbracket(\hat{U})$ the default — and any prior state — is reachable; undo is the reverse

program. (3) The optimum is a valid state (Theorem 1.3 coverage); if it is reachable by κ -permitted steps, the synthesized g is a reaching program and $\llbracket g \rrbracket$ lands there. $\square$

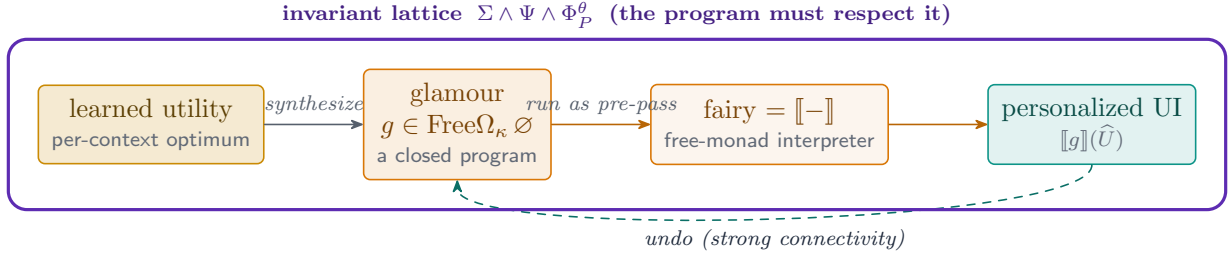


Figure 10: **The learned UI is a learned program (Proposition 10.1).** Personalization compiles the learned utility into a closed command $g \in \text{Free}_{\Omega_\kappa} \emptyset$ over capability-permitted constructors; the agent runs it by $\llbracket - \rrbracket$. Safety is that $\llbracket g \rrbracket$ stays inside the invariant lattice; reversibility is strong connectivity. “Personalize” = “synthesize and run a command.”

Remark 10.2 (The whole stack in one sentence). Reading bottom to top: a *type* is a constructor; an *element* is a cofree node over it; a *state-change* is a closed command paired against that node; a *UI* is the cofree tree; *personalization* is a synthesized free program run against it; and the invariant lattice is the predicate the program must respect. One signature $\Omega = F$ threads all six.

Part D. Calibration: where functional programming is not optimal

The functional reading is exact and buys the results above, but it is optimal only on a precise fragment. Four propositions mark the boundary; the part closes by locating it.

11 Identity lives off the functor

Proposition 11.1 (The runtime state is not a Free/Cofree value). *The cofree tree U is determined by shape and annotation up to isomorphism and carries no node identity. But the runtime globals — focus, hover, capture, drag, selection — are references to specific nodes, and the no-dangling invariant Ψ is the integrity constraint that these references point into the live document. Therefore the full runtime state is not a value of any Free/Cofree carrier; it is the pair*

$$\varrho = (U, \text{globals}), \quad \text{globals} : \{\text{focus, hover, capture, drag, selection}\} \rightarrow N,$$

a cofree tree plus a partial pointer assignment into its node-identity set $N \subseteq \mathbb{I}$. Stability of these pointers across mutations is a property no identity-quotiented (pure value) representation provides.

Proof. Free and Cofree are functors of F alone; their elements are equivalence classes under relabeling of positions, so two trees differing only by node names are equal, i.e. identity is quotiented away. The globals, however, must denote *the same* node before and after an unrelated edit elsewhere; this is data not recoverable from the shape-and-annotation class, hence external to the carrier. The

pairing with $\text{globals} : \cdot \rightarrow N$ is exactly the extra datum, and Ψ is the well-formedness of that partial map. Renaming-invariance of the carrier and pointer-stability of the globals are contradictory unless identity is tracked separately. $\square$

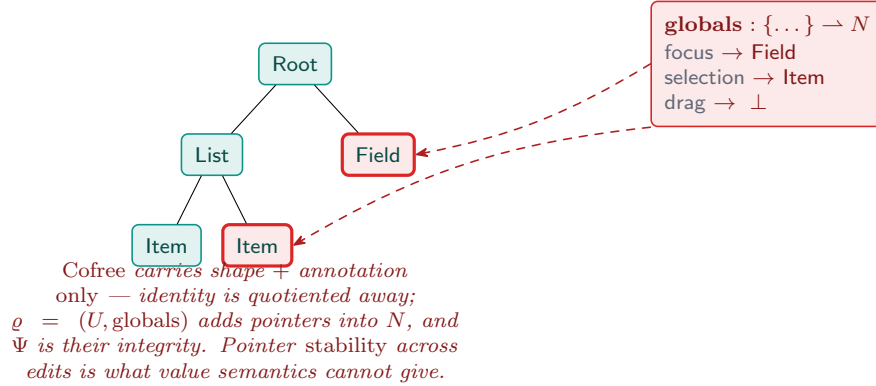


Figure 11: **Identity lives off the functor (Proposition 11.1).** The cofree tree U knows shape and state but not *which* node is which; focus, selection, and drag are references into the identity set N . The runtime state is the pair $\rho = (U, \text{globals})$, and Ψ is the no-dangling integrity of those pointers — a non-polynomial side constraint.

Remark 11.2 (Off the hierarchy, as foretold). This is the corpus’s observation that identity integrity sits off the Chomsky hierarchy, now as the functional boundary: a regular/polynomial functor captures shape; key-and-reference identity is a non-polynomial side constraint. The clean fix is an imperative node table with stable handles, which the value semantics of Free/Cofree deliberately lacks.

12 Local high-frequency mutation defeats value semantics

Proposition 12.1 (Update costs). *A single runtime event changes one node’s annotation. In the pure carrier U this requires rebuilding the path from the root to that node — $\Theta(\text{depth})$ fresh nodes per event — and a comonadic extend recomputes every node, $\Theta(\text{size})$. An in-place representation with the stable handles of Proposition 11.1 performs the same local update in $O(1)$ amortized and re-render in $O(\#\text{affected})$. Hence on the high-frequency, spatially-local events that dominate interaction (typing, dragging, hovering), value semantics is asymptotically suboptimal; the governing parameter is the frequency $\times$ locality of mutation, not the data model’s correctness.*

Proof. Persistent update of an immutable μ -tree must allocate a new spine from the mutated node to the root to preserve sharing of the untouched subtrees, which is $\Theta(\text{depth})$ nodes; this is the standard cost of functional update. A global extend visits every node by definition. An imperative table mutates the target cell in place and marks a bounded dirty set, giving the stated bounds. For a stream of m events each touching $O(1)$ nodes, the pure cost is $\Theta(m \cdot \text{depth})$ versus imperative $\Theta(m)$ plus affected re-render. $\square$

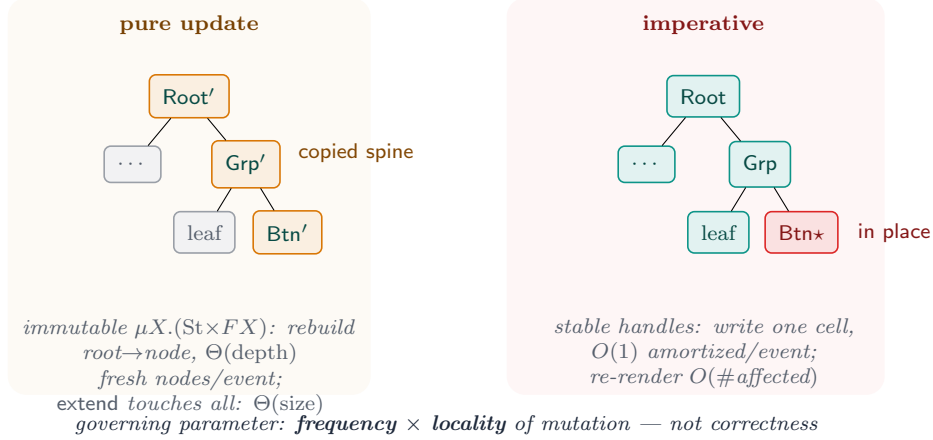


Figure 12: **Local high-frequency mutation defeats value semantics (Proposition 12.1).** One event changes one node. Persistent update must copy the spine to the root ($\Theta(\text{depth})$, and extend is $\Theta(\text{size})$); an imperative table with stable handles mutates one cell in $O(1)$. Value semantics still wins for snapshots, undo, and history — the trade is directional.

Remark 12.2 (Where value semantics still wins). The same persistence that costs on local writes *buys* cheap snapshotting, undo, structural sharing, and time-travel — precisely the operations the personalization layer (Proposition 10.1) and the editor’s history want. The trade is real and directional: pure for infrequent global moves and history, imperative for frequent local ones. A practical UI uses both, separated by exactly this boundary.

13 Observer graphs exceed trees

Proposition 13.1 (Trees, sharing DAGs, cyclic graphs). *Data-binding, two-way bindings, and observers make the dependency structure a graph, whereas μF and $\text{Cofree} F$ are trees. The corpus already lifts μF to $\text{Struct}(F)$ — a finite coalgebra with sharing edges — to express back-references and shared partials; this captures acyclic sharing (a DAG). Genuine cycles (mutual data-binding, observer loops) are not finite F -trees and not $\text{Struct}(F)$ DAGs; they require a coalgebraic (νF) or imperative (pointer-graph) representation with an explicit update/fixpoint discipline.*

Proof. A finite tree has a unique path to each node and no node reachable from itself; an observer cycle has a node depending on its own output, so it is not modeled by μF . $\text{Struct}(F)$ adds sharing as a finite coalgebra structure, representing common sub-structure (a DAG) but still terminating unfolds; a cycle makes the unfold non-terminating, leaving the $\text{Struct}(F)$ fragment. The coalgebraic νF or an explicit pointer graph with a scheduling/fixpoint rule is then required to give the loop a meaning. $\square$

14 The theory fixes what, not when

Remark 14.1 (Effects: value versus schedule). The interpretation $\llbracket - \rrbracket$ lands in an effect monad $T(R)$, and the monad/comonad laws fix the *value* a program computes and the *equalities* between programs. They do *not* fix the *schedule*: real-time deadlines, animation timing, frame pacing, input coalescing, and the interleaving of concurrent effects are outside the equational theory. Two

programs equal in T may differ in latency or perceived smoothness. The algebra is the contract for *correctness*; timing is an operational concern the functional reading deliberately abstracts, and where those constraints bind, an effect-scheduling layer outside the laws is required.

15 The stratification: the right tool per layer

Proposition 15.1 (Representation boundary). *The four observations partition the UI's structure into three regimes, each with a fitting representation:*

- (1) pure finite tree — the shape μF and the stateful tree $U = \text{Cofree } F \text{ St}$, with infrequent or global moves, history, and serialization: the **functional** carriers $\text{Free}/\text{Cofree}$ are optimal (Parts A–C);
- (2) finite acyclic sharing — back-references and shared partials: the **coalgebraic** $\text{Struct}(F)$ (a finite F -coalgebra) is the right representation;
- (3) identity-critical, cyclic, or high-frequency — focus/selection pointers (Proposition 11.1), observer graphs (Proposition 13.1), and per-frame local mutation (Proposition 12.1): a mutable, identity-bearing, **imperative** representation is optimal.

The boundaries are exact and are read off the structure itself — acyclicity, sharing, identity-dependence, mutation frequency — not chosen by taste.

Proof. Each clause is the cited result: Propositions 8.1 and 8.2 place pure trees in (1); the corpus's $\text{Struct}(F)$ lift places acyclic sharing in (2) (Proposition 13.1); and Propositions 11.1 to 13.1 place identity, cycles, and churn in (3). The criteria are structural predicates (presence of cycles, of sharing, of identity references, the frequency $\times$ locality of writes), so the assignment is determined by the structure. $\square$

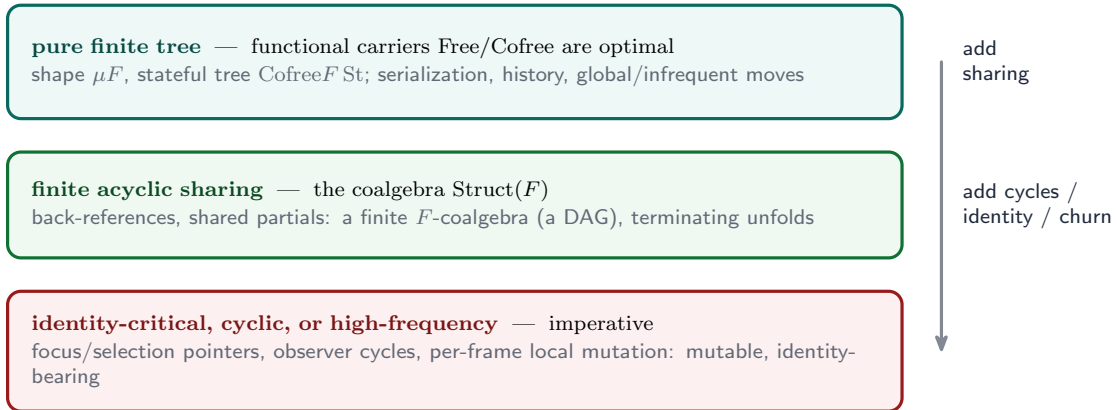


Figure 13: **The stratification: the right tool per layer (Proposition 15.1).** The boundary is read off the structure itself — acyclicity, sharing, identity-dependence, mutation frequency. Pure trees $\rightarrow$ free/cofree; acyclic sharing $\rightarrow$ the coalgebra $\text{Struct}(F)$; cycles, identity, or churn $\rightarrow$ the imperative. The functional reading of Parts A–C is exact and optimal on the top zone, and not beyond it.

Part E. Synthesis

16 The correspondence, assembled

Theorem 16.1 (The UI–DSL correspondence). *Under Assumption 1.2 and generative completeness (Theorem 1.3), for the shared signature $\Omega = F$:*

- (1) (dual carriers) *commands are the free monad $\text{Free}\Omega A$ and the stateful UI is the cofree comonad $\text{Cofree}F\text{St}$ over the same F , paired by the event loop $\text{run} = \llbracket - \rrbracket$, which is the runtime/authoring step (Theorem 3.3);*
- (2) (structure) *element types are command constructors, an arity-preserving bijection, with three further faces keyword/opcode/profile (Proposition 2.2 and Remark 2.3);*
- (3) (behaviour) *closed commands modulo behaviour are isomorphic, as a monoid, to the reachable UI state-changes — every state and event is a command and conversely (Theorem 5.1);*
- (4) (data) *an element’s attributes are an option set, its overlay the cascade $\oplus$, its diff the reconciler (Theorem 6.1); and a component with holes is a template (Definition 4.1);*
- (5) (functional support) *routing is a **Traversable** descent with **Alternative** dispatch, rendering the comonadic **extend**, serialization the parsing prism, and personalization a synthesized closed program (Propositions 7.1 to 8.2);*
- (6) (boundary) *this functional reading is optimal exactly on the pure finite-tree fragment; acyclic sharing wants the coalgebra $\text{Struct}(F)$, and identity/cycles/churn want the imperative (Proposition 15.1).*

Proof. Each clause is the cited result; together they are the assembled statement that a UI element and a DSL command are one signature $\Omega = F$ read as a responding comonad and a driving monad respectively. $\square$

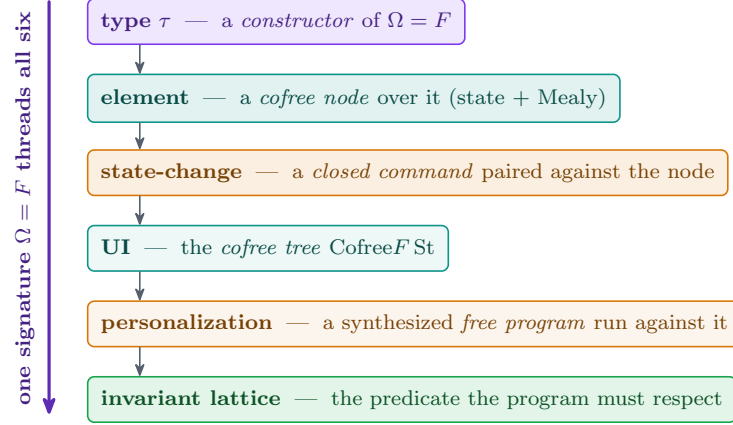


Figure 14: **The whole stack, threaded by one signature (Theorem 16.1 and Remark 10.2).** Bottom to top, each layer is a reading of the same $\Omega = F$: a type is a constructor; an element a cofree node; a state-change a closed command paired against it; the UI the cofree tree; personalization a synthesized free program; the lattice the predicate it must respect.

Remark 16.2 (What is necessary, what is contingent). *Necessary* — forced by the framework: the identification $\Omega = F$ (Proposition 2.2); the free/cofree duality and its pairing (Theorem 3.3); the behavioural isomorphism (Theorem 5.1); and attributes-as-options (Theorem 6.1). These hold however the system is built. *Contingent* — a representation choice: whether a given layer is realised in the functional carriers, the coalgebra, or the imperative. The correspondence is a theorem; the functional implementation is an option, optimal precisely where Proposition 15.1 says and not elsewhere. The single thread through both is the signature: meaning enters once, when a constructor is declared, and is read forwards as a command and backwards as an element.

A Crosswalk: one object across the companions

UI notion	Command / companion incarnation	Where
element type τ	constructor of $\Omega = F$; keyword; opcode tag; child-profile = arity	Definition 2.1 and Proposition 2.2
atomic element (leaf)	nullary constructor; bare command	Remark 4.2
composite element	internal node of $\text{Cofree}F$; command carrying sub-commands	Remark 4.2
component with a hole	template over Ω ; node of $\text{Free}\Omega A$	Definition 4.1
stateful UI tree	cofree comonad $\text{Cofree}F \text{St} = \mu X.(\text{St} \times FX)$	Definition 2.4
element (with behaviour)	Mealy node = response product = interpretation	Definition 2.6
event at a node	located command (p, e) ; choice of one response coordinate	Definition 3.1
runtime / authoring step	the $\text{Free}/\text{Cofree}$ pairing $\text{run} = \llbracket - \rrbracket$	Theorem 3.3
all state-changes	closed commands $\text{Free}\Omega \emptyset$ modulo $\approx$ (monoid iso)	Theorem 5.1
attributes / style	option set; $\text{setAttr}/\text{cascade} = \oplus$; reconcile $= \text{diff}_\pi$	Theorem 6.1
event dispatch	Traversable descent + Alternative ordered choice	Proposition 7.1
render / layout	comonadic extend ; local read extract	Proposition 8.1
serialize / address	parsing prism compose/parse on μF	Proposition 8.2
static/conditional/data-driven UI	$\text{Free}_{\text{Ap}} \subseteq \text{Free}_{\text{Sel}} \subseteq \text{Free}$ ladder	Proposition 9.1
personalization glamour	closed program $\text{Free}\Omega_\kappa \emptyset$; agent $= \llbracket - \rrbracket$	Proposition 10.1
focus/selection/drag	pointers into N ; $\varrho = (U, \text{globals})$ — off the functor	Proposition 11.1
back-reference / shared partial	coalgebra $\text{Struct}(F)$ (acyclic sharing)	Proposition 13.1
observer cycle / per-frame churn	imperative, identity-bearing representation	Proposition 15.1